

Hospital Appointment Management System

1. Project Objective

Design and implement a **Hospital Appointment Management System** using Java and Object-Oriented Programming concepts. The system should allow a hospital to maintain information about doctors and patients, schedule appointments, manage appointment status, and display appointment details.

The project should demonstrate the practical use of **classes, objects, encapsulation, inheritance, abstraction, polymorphism, interfaces, arrays, and exception handling**.

The system should provide well-defined methods that can be invoked with different inputs and should generate deterministic outputs for assessment and testing.

2. Problem Description

A hospital needs a computerized system to manage appointments between doctors and patients. The hospital has different types of doctors, such as **General Physicians, Specialists, and Surgeons**, and patients need to book appointments with the appropriate doctor.

Each doctor has a unique doctor ID, name, specialization, and consultation fee. Each patient has a unique patient ID, name, age, and contact number.

The system should maintain a collection of doctors, patients, and appointments. A patient should be able to book an appointment with an available doctor. Once an appointment is successfully booked, the doctor's availability should be updated so that the same doctor cannot be booked for the same appointment slot again.

The system should also allow an appointment to be cancelled. When an appointment is cancelled, the corresponding time slot should become available again.

The system should support different types of doctors using **inheritance and polymorphism**. A common `Doctor` class should define the basic properties and behavior, while specialized doctor classes should override appropriate methods to provide doctor-specific information.

The system should also use an interface for appointment-related operations such as booking and cancellation. Appropriate exception handling should be implemented for invalid patient IDs, invalid doctor IDs, unavailable doctors, and invalid appointment operations.

3. Functional Requirements

3.1 Add Doctor

The system should allow a new doctor to be added to the hospital.

Each doctor should contain:

- Doctor ID
- Doctor Name
- Specialization
- Consultation Fee
- Availability Status

Different types of doctors should be supported.

Example:

```
Doctor ID: D101  
Name: Dr. Amit Sharma  
Specialization: General Physician  
Fee: 500
```

3.2 Register Patient

The system should allow a new patient to be registered.

Each patient should contain:

- Patient ID
- Patient Name
- Age
- Contact Number

Example:

```
Patient ID: P101  
Name: Rahul Kumar  
Age: 35  
Contact: 9876543210
```

3.3 Search Doctor

The system should allow a doctor to be searched using the doctor ID.

If the doctor exists, the system should display the doctor's details.

If the doctor does not exist, an appropriate message or exception should be generated.

3.4 Search Patient

The system should allow a patient to be searched using the patient ID.

If the patient exists, the patient's details should be displayed.

If the patient does not exist, an appropriate message should be generated.

3.5 Book Appointment

A registered patient should be able to book an appointment with a registered doctor.

The appointment should contain:

- Appointment ID
- Patient ID
- Doctor ID
- Appointment Date
- Appointment Time
- Appointment Status
- Consultation Fee

The system should verify that:

1. The patient exists.
2. The doctor exists.
3. The requested appointment slot is available.
4. The appointment details are valid.

If all conditions are satisfied, the appointment should be created successfully.

Example:

```
Input:
Patient ID = P101
Doctor ID = D101
Date = 15-09-2026
Time = 10:00
```

Output:
Appointment booked successfully.
Appointment ID: A101

3.6 Prevent Duplicate Appointment

A doctor should not be assigned to two patients for the same date and time slot.

For example, if Dr. Amit Sharma already has an appointment at:

15-09-2026 10:00

another patient attempting to book the same doctor at the same date and time should receive:

Appointment slot is not available.

The existing appointment should remain unchanged.

3.7 Cancel Appointment

The system should allow an existing appointment to be cancelled using the appointment ID.

After cancellation:

- Appointment status should become `CANCELLED`.
- The doctor's corresponding time slot should become available.
- The appointment should remain available for record/history purposes.

Example:

Input:
Appointment ID = A101

Output:
Appointment cancelled successfully.

3.8 Display Appointment Details

The system should display complete details of an appointment.

Example output:

Appointment ID: A101
Patient ID: P101

Doctor ID: D101
Date: 15-09-2026
Time: 10:00
Status: BOOKED
Fee: 500.0

3.9 Display Patient Appointments

The system should allow all appointments associated with a particular patient to be displayed.

If the patient has multiple appointments, all corresponding appointment details should be displayed.

3.10 Display Doctor Appointments

The system should allow all appointments associated with a particular doctor to be displayed.

This operation should help determine the doctor's scheduled appointments.

3.11 Calculate Consultation Fee

The consultation fee should be obtained from the selected doctor's information.

Different doctor types may have different consultation fees.

The system should use **polymorphism** wherever appropriate to calculate or display doctor-specific information.

4. Required OOP Structure

4.1 Doctor Class

Create a base `Doctor` class containing common properties such as:

- `doctorId`
- `name`
- `specialization`
- `consultationFee`
- `available`

Suggested methods:

```
displayDoctorInfo()  
checkAvailability()  
getConsultationFee()
```

The class should demonstrate **encapsulation** by keeping data members private and providing appropriate methods to access or modify them.

4.2 Derived Doctor Classes

Create specialized classes such as:

```
GeneralPhysician  
Specialist  
Surgeon
```

These classes should inherit from the `Doctor` class.

The appropriate methods should be overridden to demonstrate **method overriding and polymorphism**.

For example:

```
GeneralPhysician  
Specialist  
Surgeon
```

may display different types of specialization information.

4.3 Patient Class

Create a `Patient` class containing:

- `patientId`
- `name`
- `age`
- `contactNumber`

Suggested methods:

```
displayPatientInfo()  
updateContactNumber()
```

4.4 Appointment Class

Create an `Appointment` class containing:

- `appointmentId`
- `patient`
- `doctor`
- `appointmentDate`
- `appointmentTime`
- `status`
- `consultationFee`

Suggested methods:

```
bookAppointment()  
cancelAppointment()  
displayAppointmentDetails()
```

The class should maintain references to the corresponding `Patient` and `Doctor` objects.

4.5 Appointment Interface

Create an interface for appointment-related operations.

For example:

```
AppointmentOperations
```

The interface may define methods such as:

```
bookAppointment()  
cancelAppointment()  
displayAppointmentDetails()
```

The appropriate class should implement this interface.

This should demonstrate the use of **interfaces and abstraction**.

4.6 Hospital Class

Create a `Hospital` class to manage:

- `Doctors`

- Patients
- Appointments

Arrays or appropriate collections may be used to store the records.

Suggested methods:

```
addDoctor()  
registerPatient()  
findDoctor()  
findPatient()  
bookAppointment()  
cancelAppointment()  
findAppointment()  
displayDoctorAppointments()  
displayPatientAppointments()
```

The implementation should demonstrate **object relationships, arrays, searching, and object management**.

5. Exception Handling

The system should handle invalid operations using appropriate exceptions.

Possible exceptions include:

DoctorNotFoundException

Generated when an invalid doctor ID is provided.

Example:

```
Input:  
Doctor ID = D999
```

```
Output:  
Doctor not found.
```

PatientNotFoundException

Generated when an invalid patient ID is provided.

Example:

```
Input:  
Patient ID = P999
```

Output:
Patient not found.

AppointmentNotAvailableException

Generated when a doctor is already booked for the requested date and time.

Example:

Output:
Appointment slot is not available.

AppointmentNotFoundException

Generated when an invalid appointment ID is used for cancellation or searching.

Example:

Output:
Appointment not found.

6. Input/Output-Based Test Cases

The following test cases should be used to verify the functionality of the system.

TC01 – Add Valid Doctor

Input

Doctor ID: D101
Name: Dr. Amit Sharma
Type: General Physician
Fee: 500

Expected Output

Doctor added successfully.

TC02 – Add Specialist Doctor

Input

Doctor ID: D102
Name: Dr. Neha Verma

Type: Specialist
Specialization: Cardiologist
Fee: 1000

Expected Output

Doctor added successfully.

TC03 – Register Valid Patient

Input

Patient ID: P101
Name: Rahul Kumar
Age: 35
Contact: 9876543210

Expected Output

Patient registered successfully.

TC04 – Search Existing Doctor

Input

Doctor ID: D101

Expected Output

Doctor ID: D101
Name: Dr. Amit Sharma
Specialization: General Physician
Fee: 500.0

TC05 – Search Non-Existing Doctor

Input

Doctor ID: D999

Expected Output

Doctor not found.

TC06 – Search Existing Patient

Input

Patient ID: P101

Expected Output

Patient ID: P101
Name: Rahul Kumar
Age: 35
Contact: 9876543210

TC07 – Book Valid Appointment

Input

Patient ID: P101
Doctor ID: D101
Date: 15-09-2026
Time: 10:00

Expected Output

Appointment booked successfully.
Appointment ID: A101
Status: BOOKED

TC08 – Verify Appointment Details

Input

Appointment ID: A101

Expected Output

Appointment ID: A101
Patient ID: P101
Doctor ID: D101
Date: 15-09-2026
Time: 10:00
Status: BOOKED
Fee: 500.0

TC09 – Attempt Duplicate Appointment

Assume D101 is already booked for 15-09-2026 10:00.

Input

Patient ID: P102
Doctor ID: D101
Date: 15-09-2026
Time: 10:00

Expected Output

Appointment slot is not available.

TC10 – Book Appointment for Different Time

Input

Patient ID: P102
Doctor ID: D101
Date: 15-09-2026
Time: 11:00

Expected Output

Appointment booked successfully.
Appointment ID: A102
Status: BOOKED

TC11 – Book Appointment with Invalid Patient

Input

Patient ID: P999
Doctor ID: D101
Date: 15-09-2026
Time: 12:00

Expected Output

Patient not found.

TC12 – Book Appointment with Invalid Doctor

Input

Patient ID: P101
Doctor ID: D999
Date: 15-09-2026
Time: 12:00

Expected Output

Doctor not found.

TC13 – Cancel Existing Appointment

Input

Appointment ID: A101

Expected Output

Appointment cancelled successfully.
Status: CANCELLED

TC14 – Verify Cancelled Appointment

Input

Appointment ID: A101

Expected Output

Appointment ID: A101
Status: CANCELLED

TC15 – Book Previously Released Slot

After appointment A101 is cancelled, the same doctor and time slot should become available.

Input

Patient ID: P102
Doctor ID: D101
Date: 15-09-2026
Time: 10:00

Expected Output

Appointment booked successfully.

Appointment ID: A103
Status: BOOKED

TC16 – Cancel Non-Existing Appointment

Input

Appointment ID: A999

Expected Output

Appointment not found.

TC17 – Display Patient Appointments

Input

Patient ID: P101

Expected Output

Patient ID: P101
Appointments:
A101 - D101 - 15-09-2026 - 10:00 - CANCELLED

TC18 – Display Doctor Appointments

Input

Doctor ID: D101

Expected Output

Doctor ID: D101
Appointments:
A102 - P102 - 15-09-2026 - 11:00 - BOOKED

TC19 – Verify Polymorphic Doctor Information

Input

Doctor Type: Specialist
Doctor ID: D102

Expected Output

Doctor ID: D102
Name: Dr. Neha Verma
Type: Specialist
Specialization: Cardiologist
Fee: 1000.0

TC20 – Invalid Age During Patient Registration

Input

Patient ID: P103
Name: Amit Kumar
Age: -5
Contact: 9876500000

Expected Output

Invalid patient age.

7. Consolidated Test Case Table

Test Case	Operation	Input Condition	Expected Output
TC01	Add Doctor	Valid doctor details	Doctor added successfully
TC02	Add Specialist	Valid specialist details	Doctor added successfully
TC03	Register Patient	Valid patient details	Patient registered successfully
TC04	Search Doctor	Existing ID	Doctor details displayed
TC05	Search Doctor	Invalid ID	Doctor not found
TC06	Search Patient	Existing ID	Patient details displayed
TC07	Book Appointment	Valid patient, doctor and slot	Appointment booked
TC08	View Appointment	Existing appointment ID	Appointment details displayed
TC09	Duplicate Booking	Same doctor/date/time	Slot unavailable
TC10	Different Slot	Available time	Appointment booked
TC11	Invalid Patient	Invalid patient ID	Patient not found
TC12	Invalid Doctor	Invalid doctor ID	Doctor not found
TC13	Cancel Appointment	Existing appointment	Appointment cancelled
TC14	Verify Cancellation	Cancelled appointment	Status = CANCELLED
TC15	Reuse Slot	Previously cancelled slot	Appointment booked
TC16	Cancel Appointment	Invalid appointment ID	Appointment not found
TC17	Patient Appointments	Existing patient	Patient appointments displayed
TC18	Doctor Appointments	Existing doctor	Doctor appointments displayed
TC19	Polymorphism	Specialist doctor	Specialized information displayed
TC20	Invalid Patient Data	Negative age	Invalid patient age

8. Assessment Requirements

For automated assessment, the implementation should follow the prescribed **package names**, **class names**, **method names**, **method parameters**, and **return types** exactly. The assessment system should be able to create objects of the required classes and invoke the specified methods directly.

The implementation should demonstrate:

- Classes and Objects
- Encapsulation
- Constructors
- Arrays
- Inheritance
- Method Overriding
- Polymorphism
- Abstraction
- Interfaces
- Object Association
- Searching
- Exception Handling
- Input Validation

The test cases should verify both **successful and unsuccessful operations**, including valid registration, searching, appointment booking, duplicate booking prevention, cancellation, slot reuse, invalid IDs, and invalid input values.

This project is deliberately structured so that the **functional behavior can be checked through fixed Input/Output test cases**, similar to the assessment-oriented structure of your reference material.